遗传算法代码实践小组实验报告

一、实验目的与代码来源

本实验运行公开遗传算法示例，理解初始化、适应度评价、选择、交叉、变异和代际更新流程，再在统一代码和测试问题下，采用单因素对照研究交叉概率与不同算子的作用。

原始代码选自DEAP官方`examples/ga/onemax_short.py`。

二、统一测试问题与基本流程

2.1 OneMax问题的完整定义

OneMax是一个二进制优化问题：在长度固定的0/1串中，让数字1的数量尽可能多。设串长为n，一个候选解记为$x=(x_1,x_2,\ldots,x_n)$，每个x_i只能取0或1；所有这样的串均为合法解，没有额外约束。目标函数定义为：

$$\text{最大化 } f(x) = x_1 + x_2 + \ldots + x_n$$
$$\text{约束：} x_i \in \{0,1\},\ i=1,\ldots,n$$

例如，n=8时，x=10110010中有4个1，因此f(x)=4；将其中一个0改为1后，适应度变为5。最优解为11111111，最优值为8。

本实验固定n=100，搜索空间共有2^{100}个不同的二进制串，适应度为0到100之间的整数。唯一全局最优解是100个1组成的串，最优值为100；含99个1的串得99分。只要串中还有0，就能通过翻转该位使目标值增加1，因此在单个位翻转的邻域下，不存在最优解以外的局部极大值。各位的贡献独立，每个1都贡献1分。

选择该问题是因为最优值已知，便于核验算法是否成功，并在统一条件下比较收敛速度。实验仍从随机种群出发执行完整遗传算法，记录发现最优解的过程。该问题不能代表复杂约束、位点相互依赖或存在多个局部极大值的问题。

2.2 遗传算法的基本概念

遗传算法（Genetic Algorithm，GA）是一种维护多个候选解、反复筛选并产生新解的随机搜索方法。本实验使用以下定义：

概念	在本实验中的含义
个体 / 染色体	一个长度100的二进制串，代表一个候选解
基因 / 位点	串中的一个位置及其0或1取值；等位基因指该位置可能取的值
种群	同一代的N个候选解，本实验N=80
适应度	候选解的评分，直接取OneMax目标函数f(x)，越大越好
代	一轮种群更新；随机初始种群记为第0代，更新后为第1代，依此类推
遗传算子	选择父本、重组父本或改变基因的规则，分别称为选择、交叉、变异
精英保留	将上一代适应度最高的1个个体原样放入下一代，避免当前最好解丢失
收敛速度	本报告用首次发现最优解的代数衡量，代数越少表示越快

选择压力指选择规则偏向高适应度个体的程度；压力较强通常会加快优良个体扩散，也可能使种群更快失去不同基因取值。种群多样性指个体在各位取值上的差异，具体指标在4.3节定义。

2.3 本实验的迭代流程

```
随机初始化80个染色体 → 计算适应度
  ↓
选择80个父本 → 两两按Pc交叉 → 按位变异 → 用上一代精英替换1个子代
  ↓
评价新一代 → 记录最优值、平均值、多样性、是否首次达到100
  ↓
固定运行100代（达到100也继续）→ 汇总30个种子的统计结果
```

交叉概率Pc定义为一对父本执行交叉的概率。Pc=0.8表示每对父本有80%的概率执行交叉；80个父本配成40对时，平均约32对触发交叉，但每代实际数量可以波动。未触发时先复制父本，仍可继续发生变异。均匀交叉触发后，各位另以0.5概率交换。

变异概率Pm定义为一个子代的某一位翻转的概率，翻转即0变1或1变0。基本变异的Pm=0.01意味着100位个体平均翻转1位，实际可以为0位或多位。本扩展实现对各位独立变异，没有再套一层个体变异概率；随后用未变异的精英替换一个子代。

三、原始代码运行结果

原始示例固定seed=64、种群300、100位、运行40代；采用三元锦标赛、两点交叉，Pc=0.5，个体变异概率0.2，触发变异后每位翻转概率0.05；HallOfFame只保存历史最优，不自动把精英注入种群。

实际运行的历史最优值为 **100/100**；末代最大值为100，末代平均值为98.833。完整40代输出与最优串已记录。以下参数实验使用统一扩展基准，保持各对照组预算一致。

四、受控实验设计与算子任务

4.1 统一设置与公平比较

项目	统一设置
染色体与目标	100位二进制；最大化1的个数，最优100
种群与迭代	80个体、100代，全部运行固定预算
基准组合	Pc=0.8、三元锦标赛、两点交叉、基本变异
变异与精英	基本逐位Pm=0.01；每代保留1个精英，不参与变异
重复与种子	每配置30次，种子20261023至20261052
评价预算	初代及每代均评价80个体，每次8080次OneMax评价

不同配置对相同种子使用完全相同初始种群，并保存SHA256进行核对。初始化、选择、交叉和变异采用四条独立随机数流；算法变化后后续种群会不同，相同种子并不意味着所有后续随机事件完全一致。所有组均不提前停止。随机种子是初始化伪随机数生成器的整数，用于重复同一随机实验；SHA256是从数据计算的固定长度校验值，此处用于核对初始种群文件内容是否一致。

4.2 算子任务划分

算子实验按以下任务编号组织，组长可按实际人数分配，成员选择的任务编号不重复。

任务编号	算子	本次实现
S1	轮盘赌选择	按非负适应度占比采样，全零时均匀采样
S2	锦标赛选择	随机抽3个候选，取适应度最高者；允许重复抽样
S3	排序选择	线性排序，选择压力s=1.7，同适应度使用平均秩
C1	单点交叉	选一个内部断点，交换后缀
C2	两点交叉	选两个不同内部断点，交换中段
C3	均匀交叉	触发交叉后，每位以0.5概率交换
M1	基本变异	每位固定Pm=0.01独立翻转
M2	自适应变异	根据上一代多样性调整逐位Pm，公式如下

4.3 八种算子的具体定义

选择算子。 每代从当前种群有放回地选出80个父本，同一个个体可以被选多次。记第i个个体的适应度为f_i，种群规模为N。

1. 轮盘赌选择：每次选中个体i的概率为$q_i=f_i/\Sigma f_k$。例如适应度为2、3、5时，选择概率分别为0.2、0.3、0.5；全部为0时，各个体概率均为1/N。
2. 三元锦标赛选择：每次均匀随机抽取3个候选，允许重复抽取，从中选择适应度最高者。最高分并列时取本次抽样中先出现的候选。重复80次形成父本集合。
3. 线性排序选择：按适应度从低到高赋予秩r_i，最差为1、最好为N，同分者使用所占名次的平均值。例如排序后的适应度为1、2、2、4时，秩为1、2.5、2.5、4。定义$q_i=(2-s)/N+2(r_i-1)(s-1)/[N(N-1)]$，本实验$s=1.7$。s在[1,2]内控制选择压力，s=1时均匀选择；无并列时最好者和最差者的概率分别为s/N和(2−s)/N。

交叉算子。 对每一对父本先按Pc决定是否交叉，触发后产生两个互补交换的子代。下述断点k表示前k位之后的位置，内部断点从1至n−1中均匀抽取。

1. 单点交叉：抽取一个内部断点，交换断点后的全部位。例如父本1111|0000和0000|1111在第4位后交换，得到11111111和00000000。
2. 两点交叉：抽取两个不同内部断点a<b，交换第a+1至第b位，其他位置保持。例如父本11|1100|00和00|0011|11在第2、6位后切开，交换中段得到11001100和00110011。
3. 均匀交叉：对每个位点独立产生一个交换决定，概率为0.5；决定交换时，两子代在该位互换父本取值，否则各自保留对应父本取值。因此一次交叉可以在多个分散位置交换。

变异算子。 交叉后对子代的每一位独立作翻转决定，基本变异固定Pm=1/n=0.01。自适应变异则根据当前种群的多样性，为下一代生成过程计算Pm。记x_{ij}为第i个个体在第j位的0/1取值，第j位取1的比例为$p_j=(\Sigma_i x_{ij})/N$，定义：

```
D = [Σ_j 4p_j(1-p_j)] / n
自适应Pm = 0.01 + 0.02(1-D)
```

这里求和遍历全部n个位点，D是各位差异程度的平均值，范围为[0,1]。当所有个体完全相同时，每位p_j均为0或1，D=0；当每位恰有一半个体取1时，D=1。因此自适应Pm范围为[0.01,0.03]，D=1时取0.01，D=0时取0.03，多样

性下降时增加扰动。这是本作业定义的一种自适应规则，不代表所有自适应变异，也不保证更优。

4.4 单因素对照方案

交叉概率取0、0.2、0.5、0.8、1.0，其余用基准设置。选择、交叉、变异算子对照分别只替换对应算子，其余保持Pc=0.8与基准配置。基准被多张表复用，共10种唯一配置，而非重复运行并挑选有利结果。

五、结果与指标

每次运行把某一代种群中最大的f(x)记为该代最优适应度，末代最优值为第100代的该值。成功指第0至100代中至少一次出现f(x)=100，成功率为成功次数除以30。成功代数指首次出现100的代数，其均值只对成功运行计算；没有成功运行时用‘—’表示。

终值及惩罚代数均报告30次运行的均值±样本标准差（SD）。对于30个观测值y_i，均值$\bar{y}=(\Sigma y_i)/30$，$SD=\sqrt{[\Sigma(y_i-\bar{y})^2/29]}$；SD越大表示不同种子的结果波动越大，表中的‘±’表示离散程度，不表示结果必定落在该范围内。

为计入未成功的运行，定义惩罚到达代数：成功取首次成功代数，未成功取预算上限100再加1，即101，越小越好。例如两次运行分别在第40代成功、100代内未成功，其惩罚代数均值为(40+101)/2=70.5。它是人为定义的预算惩罚指标，不是真实的无条件期望收敛时间。

5.1 交叉概率对照

配置	末代最优均值±SD	成功次数	成功代数均值	惩罚代数均值±SD
Pc=0.0	99.67±0.48	20/30	92.80	95.53±5.84
Pc=0.2	100.00±0.00	30/30	65.97	65.97±9.10
Pc=0.5	100.00±0.00	30/30	51.57	51.57±5.16
基准：Pc=0.8	100.00±0.00	30/30	41.23	41.23±5.78
Pc=1.0	100.00±0.00	30/30	37.43	37.43±3.86

5.2 选择算子对照

配置	末代最优均值±SD	成功次数	成功代数均值	惩罚代数均值±SD
锦标赛选择（基准）	100.00±0.00	30/30	41.23	41.23±5.78
轮盘赌选择	86.83±2.10	0/30	—	101.00±0.00
线性排序选择	99.73±0.45	22/30	85.27	89.47±10.43

5.3 交叉算子对照

配置	末代最优均值±SD	成功次数	成功代数均值	惩罚代数均值±SD
两点交叉（基准）	100.00±0.00	30/30	41.23	41.23±5.78
单点交叉	100.00±0.00	30/30	47.37	47.37±4.00
均匀交叉	100.00±0.00	30/30	27.03	27.03±4.13

5.4 变异算子对照

配置	末代最优均值±SD	成功次数	成功代数均值	惩罚代数均值±SD
基本变异（基准）	100.00±0.00	30/30	41.23	41.23±5.78
自适应变异	99.13±0.68	9/30	78.22	94.17±13.93

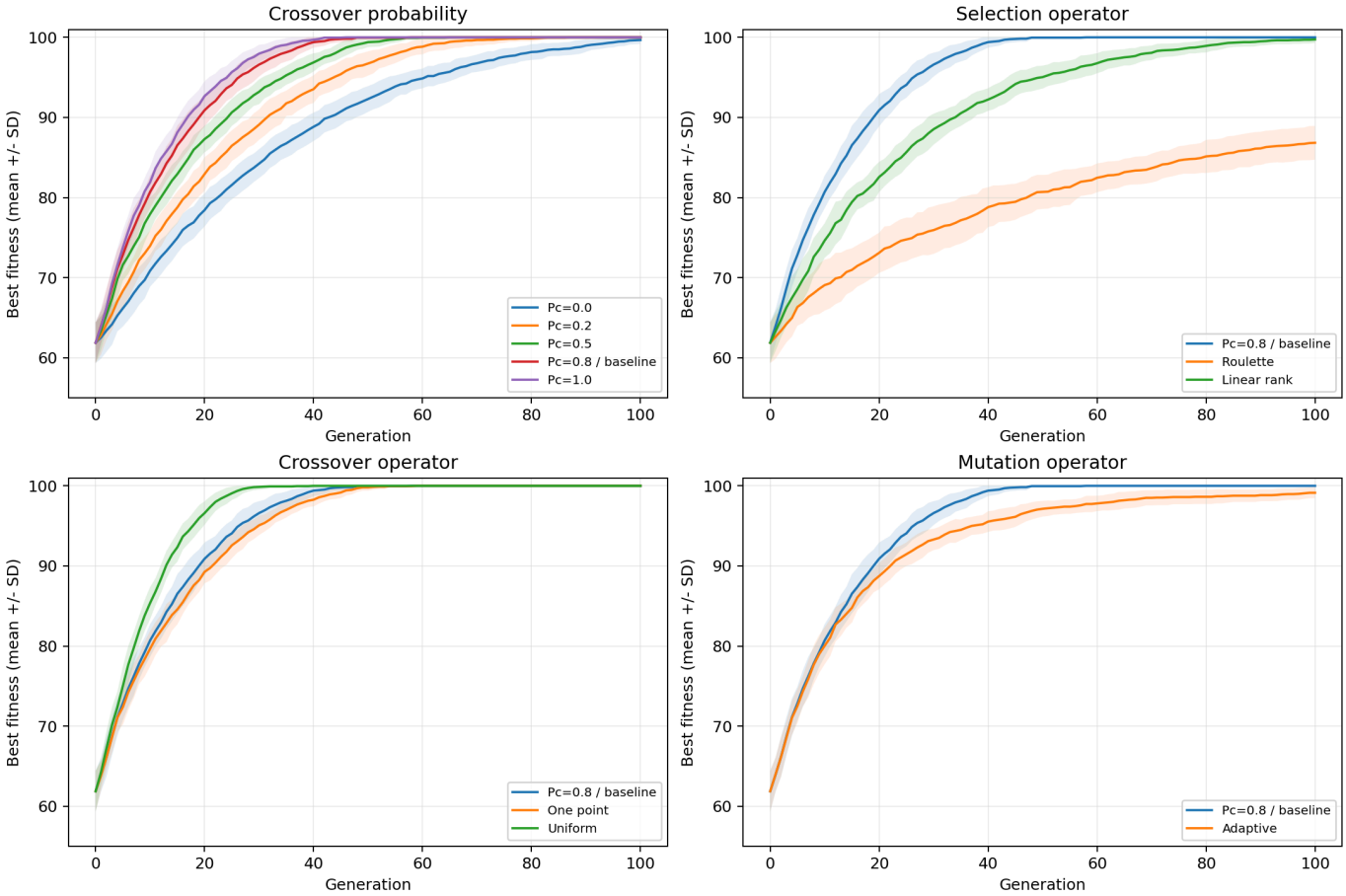


图1：曲线为多次运行的最优适应度均值，阴影为±1个样本标准差，不是置信区间。

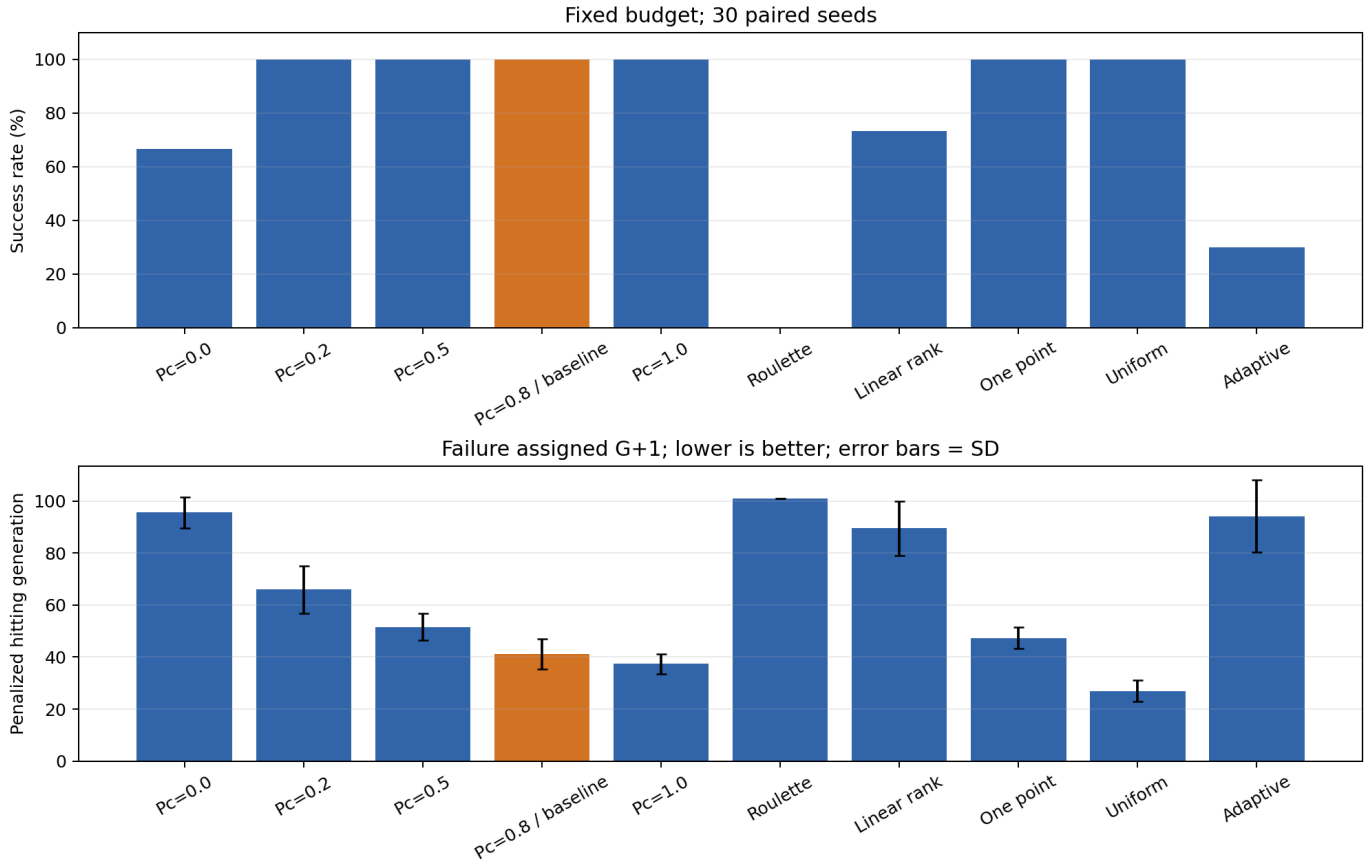


图2：固定预算下的成功率和惩罚到达代数；误差棒为样本标准差。

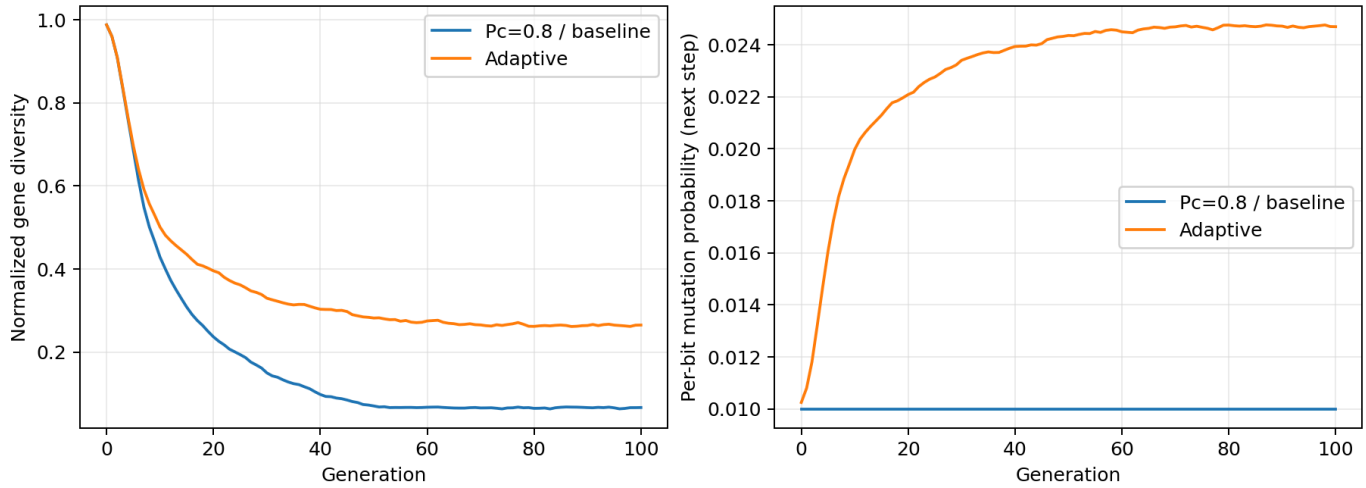


图3：基准与自适应变异的平均多样性和逐位变异概率；第100代概率是按末代状态计算，未继续执行。

5.5 同种子配对差异

对每个种子计算‘该配置惩罚代数−基准惩罚代数’，负数表示更快。bootstrap在这里指有放回重采样：每次从30个种子的配对差值中随机抽30个，计算其均值，重复10000次，固定统计种子314159。将这10000个均值排序，取2.5%和97.5%分位数作为百分位95%区间。这些区间用于探索性描述，未做多重比较校正；区间跨0时不据均值排序断言谁更优。

配置	配对均值差/代	bootstrap 95%区间
Pc=0.0	54.30	[51.70, 56.83]
Pc=0.2	24.73	[21.27, 28.53]

配置	配对均值差/代	bootstrap 95%区间
Pc=0.5	10.33	[7.90, 13.07]
Pc=1.0	-3.80	[-6.20, -1.40]
轮盘赌选择	59.77	[57.70, 61.77]
线性排序选择	48.23	[43.97, 52.33]
单点交叉	6.13	[3.47, 8.67]
均匀交叉	-14.20	[-16.60, -11.67]
自适应变异	52.93	[47.17, 58.00]

六、分析、发现的问题与结论

交叉概率。 本次Pc组中，Pc=1.0的惩罚代数均值最低，为37.43。交叉可以组合不同个体中已有的有利位点，但不会凭空产生缺失的等位基因；Pc=0仍可通过选择和变异改进。较大Pc也会打散已有组合，不能仅凭参数大小判断结果，更不能把本次最佳值称为普遍最优Pc。

选择算子。 基准锦标赛成功30/30，轮盘赌0/30，排序选择22/30。在OneMax后期，大多数个体的适应度接近，轮盘赌的相对权重差缩小，选择压力可能不足；锦标赛保持较强筛选，线性排序通过秩控制压力。这是具体压力参数的对照，不能证明任意锦标赛都优于排序选择。

交叉算子。 本次交叉组中均匀交叉的惩罚代数均值最低。单点与两点交叉保留连续片段，均匀交叉更细粒度重组位点。OneMax各位贡献独立，不存在必须保持的相邻联动结构，因而这种结果不能直接推广到位点间依赖很强的问题。

变异算子。 自适应组成功9/30，惩罚代数均值94.17，基准为41.23。提高变异率能补充多样性，但在接近全1时也会把更多有利位点翻回0。本规则只根据多样性提高Pm，没有根据距离最优的程度减小扰动；自适应并不等于性能必然提高。

指标选择问题。 多组实验末代最优值可能同为100，单看终值会掩盖收敛速度差异；只统计成功代数又会忽略失败，因此同时报告成功率、成功代数和带失败惩罚的指标。

可比性与局限。 全部组共享初始种群、预算、精英数和单一目标函数。运行耗时另存于CSV，但NumPy向量化程度和算子实现开销不同，本文不把单次墙钟时间当作算法理论复杂度。30个种子只能支持当前问题和预算下的经验结论，未测试算子组合交互、复杂约束问题或多峰函数。

本次实践验证了GA的完整流程，并完成5个Pc取值、3种选择、3种交叉和2种变异的单因素对照。最终结果表明应结合选择压力、重组方式、变异强度和多样性理解性能，避免只报告一次最优运行。

实验代码

DEAP 官方原始示例（vendor/onemax_short.py）

```
#    This file is part of DEAP.
#
#    DEAP is free software: you can redistribute it and/or modify
#    it under the terms of the GNU Lesser General Public License as
```

```
#    published by the Free Software Foundation, either version 3 of
#    the License, or (at your option) any later version.
#
#    DEAP is distributed in the hope that it will be useful,
#    but WITHOUT ANY WARRANTY; without even the implied warranty of
#    MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
#    GNU Lesser General Public License for more details.
#
#    You should have received a copy of the GNU Lesser General Public
#    License along with DEAP. If not, see <http://www.gnu.org/licenses/>.

import array
import random

import numpy

from deap import algorithms
from deap import base
from deap import creator
from deap import tools

creator.create("FitnessMax", base.Fitness, weights=(1.0,))
creator.create("Individual", array.array, typecode='b', fitness=creator.FitnessMax)

toolbox = base.Toolbox()

# Attribute generator
toolbox.register("attr_bool", random.randint, 0, 1)

# Structure initializers
toolbox.register("individual", tools.initRepeat, creator.Individual, toolbox.attr_bool, 100)
toolbox.register("population", tools.initRepeat, list, toolbox.individual)

def evalOneMax(individual):
    return sum(individual),

toolbox.register("evaluate", evalOneMax)
toolbox.register("mate", tools.cxTwoPoint)
toolbox.register("mutate", tools.mutFlipBit, indpb=0.05)
toolbox.register("select", tools.selTournament, tournsize=3)

def main():
    random.seed(64)

    pop = toolbox.population(n=300)
    hof = tools.HallOfFame(1)
    stats = tools.Statistics(lambda ind: ind.fitness.values)
    stats.register("avg", numpy.mean)
    stats.register("std", numpy.std)
    stats.register("min", numpy.min)
    stats.register("max", numpy.max)

    pop, log = algorithms.eaSimple(pop, toolbox, cxpb=0.5, mutpb=0.2, ngen=40,
                                   stats=stats, halloffame=hof, verbose=True)

    return pop, log, hof

if __name__ == "__main__":
    main()
```

遗传算法与八种算子实现（ga.py）

```
"""Controlled OneMax GA implementation; operator definitions follow DEAP examples.

The upstream source is retained in vendor/. This NumPy implementation is original
and intentionally evaluates the whole population each generation for equal budgets.
"""
from dataclasses import asdict, dataclass
from time import perf_counter

import numpy as np


@dataclass(frozen=True)
class Config:
    length: int = 100
    population: int = 80
    generations: int = 100
    pc: float = 0.8
    selection: str = "tournament"
    crossover: str = "two_point"
    mutation: str = "basic"
    tournament_size: int = 3
    rank_pressure: float = 1.7
    elitism: int = 1

    def validate(self):
        if self.length < 3 or self.population < 4 or self.population % 2:
            raise ValueError("length >= 3; population >= 4 and even")
        if self.generations < 1 or not 0 <= self.pc <= 1:
            raise ValueError("generations >= 1 and pc in [0,1]")
        if self.selection not in {"roulette", "tournament", "rank"}:
            raise ValueError("unknown selection")
        if self.crossover not in {"one_point", "two_point", "uniform"}:
            raise ValueError("unknown crossover")
        if self.mutation not in {"basic", "adaptive"}:
            raise ValueError("unknown mutation")
        if self.tournament_size < 2 or not 1 <= self.rank_pressure <= 2:
            raise ValueError("tournament_size >= 2; rank_pressure in [1,2]")
        if not 0 <= self.elitism < self.population:
            raise ValueError("elitism in [0,population)")


def select_indices(fitness, config, rng):
    n = len(fitness)
    if config.selection == "tournament":
        candidates = rng.integers(n, size=(n, config.tournament_size))
        return candidates[np.arange(n), fitness[candidates].argmax(axis=1)]
    if config.selection == "roulette":
        probabilities = fitness / fitness.sum() if fitness.sum() else np.full(n, 1 / n)
    else:
        # Equal fitness receives the same mean rank, avoiding arbitrary tie bias.
        _, inverse, counts = np.unique(fitness, return_inverse=True, return_counts=True)
        rank = (np.cumsum(counts) - (counts - 1) / 2)[inverse]
        s = config.rank_pressure
        probabilities = (2 - s) / n + 2 * (rank - 1) * (s - 1) / (n * (n - 1))
        probabilities /= probabilities.sum()
    return rng.choice(n, size=n, replace=True, p=probabilities)


def crossover(parents, kind, pc, rng):
```

```
    children = parents.copy()
    pairs, length = len(parents) // 2, parents.shape[1]
    apply = rng.random(pairs) < pc
    loci = np.arange(length)[None, :]
    if kind == "one_point":
        cuts = rng.integers(1, length, size=(pairs, 1))
        mask = loci >= cuts
    elif kind == "two_point":
        a = rng.integers(1, length, size=(pairs, 1))
        b = rng.integers(1, length - 1, size=(pairs, 1))
        b += b >= a
        mask = (loci >= np.minimum(a, b)) & (loci < np.maximum(a, b))
    elif kind == "uniform":
        mask = rng.random((pairs, length)) < 0.5
    else:
        raise ValueError(kind)
    mask &= apply[:, None]
    left, right = parents[::2], parents[1::2]
    children[::2] = np.where(mask, right, left)
    children[1::2] = np.where(mask, left, right)
    return children


def diversity(population):
    frequency = population.mean(axis=0)
    return float(np.mean(4 * frequency * (1 - frequency)))


def mutation_probability(config, current_diversity):
    base = 1 / config.length
    return base if config.mutation == "basic" else base + (3 / config.length - base) * (1 - current_diversity)


def run(config=Config(), seed=0):
    config.validate()
    start = perf_counter()
    init_rng, select_rng, cross_rng, mutate_rng = [np.random.default_rng(s)
        for s in np.random.SeedSequence(seed).spawn(4)]
    population = init_rng.integers(0, 2, (config.population, config.length), dtype=np.uint8)
    initial_population = population.copy()
    fitness = population.sum(axis=1)
    first_hit = 0 if fitness.max() == config.length else None
    history = []
    for generation in range(config.generations + 1):
        d = diversity(population)
        pm = mutation_probability(config, d)
        history.append({"generation": generation, "best": int(fitness.max()),
                        "mean": float(fitness.mean()), "diversity": d, "pm": pm})
        if generation == config.generations:
            break
        elites = population[np.argsort(fitness, kind="stable")[-config.elitism:]].copy() if config.elitism else
None
        parents = population[select_indices(fitness, config, select_rng)]
        children = crossover(parents, config.crossover, config.pc, cross_rng)
        children ^= (mutate_rng.random(children.shape) < pm).astype(np.uint8)
        # Elites survive unchanged; evaluate a full N-member new population.
        if config.elitism:
            children[:config.elitism] = elites
        population = children
        fitness = population.sum(axis=1)
        if first_hit is None and fitness.max() == config.length:
            first_hit = generation + 1
```

```
    best = population[fitness.argmax()]
    assert int(best.sum()) == int(fitness.max())
    assert np.isin(best, [0, 1]).all()
    return {"config": asdict(config), "seed": seed, "initial_population": initial_population.tolist(),
            "best_genome": "".join(map(str, best.tolist())), "final_best": int(fitness.max()),
            "final_mean": float(fitness.mean()), "success": first_hit is not None,
            "first_hit": first_hit, "penalized_hit": first_hit if first_hit is not None else config.generations
+ 1,
            "evaluations": config.population * (config.generations + 1),
            "elapsed_seconds": perf_counter() - start, "history": history}
```

受控实验运行脚本（run_experiments.py）

```
"""Run the archived official example, then paired, controlled operator experiments."""
import argparse
from contextlib import redirect_stdout
from dataclasses import asdict, replace
from datetime import datetime
import csv
import hashlib
import importlib.metadata
import importlib.util
import json
from pathlib import Path
import platform
import sys

from ga import Config, run

ROOT = Path(__file__).resolve().parent


def configurations():
    base = Config()
    items = [(f"pc_{pc:.1f}", "pc", replace(base, pc=pc)) for pc in (0.0, 0.2, 0.5, 0.8, 1.0)]
    items += [("select_" + s, "selection", replace(base, selection=s)) for s in ("roulette", "rank")]
    items += [("cross_" + c, "crossover", replace(base, crossover=c)) for c in ("one_point", "uniform")]
    items += [("mut_adaptive", "mutation", replace(base, mutation="adaptive"))]
    return items


def official_example(output):
    spec = importlib.util.spec_from_file_location("official_onemax", ROOT / "vendor/onemax_short.py")
    module = importlib.util.module_from_spec(spec)
    spec.loader.exec_module(module)
    with (output / "original_stdout.txt").open("w") as stream, redirect_stdout(stream):
        pop, log, hof = module.main()
    result = {"best_fitness": float(hof[0].fitness.values[0]), "best_genome": list(hof[0]),
              "history": list(log), "seed": 64, "population": 300, "generations": 40,
              "pc": 0.5, "individual_mutation_probability": 0.2, "bit_flip_probability": 0.05,
              "selection": "tournament3", "crossover": "two_point"}
    (output / "original_result.json").write_text(json.dumps(result, indent=2))


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--output", type=Path, default=ROOT / "results")
    parser.add_argument("--repeats", type=int, default=30)
    parser.add_argument("--first-seed", type=int, default=20261023)
    args = parser.parse_args()
```

```
    if args.repeats < 2:
        parser.error("repeats >= 2")
    args.output.mkdir(parents=True, exist_ok=False)
    official_example(args.output)
    seeds = list(range(args.first_seed, args.first_seed + args.repeats))
    manifest = {"created_at": datetime.now().astimezone().isoformat(), "python": sys.version,
                "platform": platform.platform(), "processor": platform.processor(),
                "packages": {name: importlib.metadata.version(name) for name in ("numpy", "matplotlib",
"deap")},
                "seeds": seeds, "baseline_id": "pc_0.8", "fixed_generations": True,
                "configurations": [{"id": key, "factor": factor, **asdict(config)}
                                   for key, factor, config in configurations()]}
    (args.output / "manifest.json").write_text(json.dumps(manifest, ensure_ascii=False, indent=2))
    columns = ["config_id", "factor", "seed", "final_best", "final_mean", "success", "first_hit",
               "penalized_hit", "evaluations", "elapsed_seconds", "initial_sha256", "best_genome"]
    initial_hashes = {}
    with (args.output / "runs.jsonl").open("w") as raw, (args.output / "runs.csv").open("w", newline="") as
table:
        writer = csv.DictWriter(table, fieldnames=columns)
        writer.writeheader()
        for key, factor, config in configurations():
            for seed in seeds:
                result = run(config, seed)
                initial = json.dumps(result.pop("initial_population"), separators=(",", ":"))
                digest = hashlib.sha256(initial.encode()).hexdigest()
                if seed in initial_hashes:
                    assert digest == initial_hashes[seed]
                else:
                    initial_hashes[seed] = digest
                    (args.output / f"initial_seed_{seed}.json").write_text(initial)
                result.update(config_id=key, factor=factor, initial_sha256=digest)
                raw.write(json.dumps(result) + "\n")
                writer.writerow({name: result.get(name) for name in columns})
            raw.flush()
            print(f"completed {key}: {len(seeds)} runs", flush=True)


if __name__ == "__main__":
    main()
```

统计与绘图脚本（plot.py）

```
"""Produce auditable summaries, figures and a Chinese group report from raw runs."""
import csv
import json
import os
from pathlib import Path

import numpy as np

LABELS = {"pc_0.0": "Pc=0.0", "pc_0.2": "Pc=0.2", "pc_0.5": "Pc=0.5",
          "pc_0.8": "Pc=0.8 / baseline", "pc_1.0": "Pc=1.0",
          "select_roulette": "Roulette", "select_rank": "Linear rank",
          "cross_one_point": "One point", "cross_uniform": "Uniform", "mut_adaptive": "Adaptive"}
CHINESE = {"pc_0.0": "Pc=0.0", "pc_0.2": "Pc=0.2", "pc_0.5": "Pc=0.5",
           "pc_0.8": "基准: Pc=0.8", "pc_1.0": "Pc=1.0", "select_roulette": "轮盘赌选择",
           "select_rank": "线性排序选择", "cross_one_point": "单点交叉",
           "cross_uniform": "均匀交叉", "mut_adaptive": "自适应变异"}
```

```
def summarize(output):
    os.environ.setdefault("MPLCONFIGDIR", "/tmp/ga-lab-matplotlib")
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt
    manifest = json.loads((output / "manifest.json").read_text())
    records = [json.loads(line) for line in (output / "runs.jsonl").read_text().splitlines()]
    groups = {config["id"]: [r for r in records if r["config_id"] == config["id"]]
              for config in manifest["configurations"]}
    baseline = groups[manifest["baseline_id"]]
    rows = []
    bootstrap_rng = np.random.default_rng(314159)
    resamples = bootstrap_rng.integers(len(baseline), size=(10000, len(baseline)))
    for key, runs in groups.items():
        assert [r["seed"] for r in runs] == [r["seed"] for r in baseline]
        assert [r["initial_sha256"] for r in runs] == [r["initial_sha256"] for r in baseline]
        final = np.array([r["final_best"] for r in runs])
        hits = np.array([r["penalized_hit"] for r in runs])
        delta = hits - np.array([r["penalized_hit"] for r in baseline])
        low, high = np.quantile(delta[resamples].mean(axis=1), [0.025, 0.975])
        successful = [r["first_hit"] for r in runs if r["success"]]
        row = {"config_id": key, "n": len(runs), "best_mean": float(final.mean()),
               "best_std": float(final.std(ddof=1)), "best_min": int(final.min()), "best_max":
int(final.max()),
               "successes": len(successful), "success_rate": len(successful) / len(runs),
               "hit_mean_success_only": float(np.mean(successful)) if successful else None,
               "penalized_hit_mean": float(hits.mean()), "penalized_hit_std": float(hits.std(ddof=1)),
               "paired_delta_mean": float(delta.mean()), "paired_delta_ci_low": float(low),
               "paired_delta_ci_high": float(high), "seconds_mean": float(np.mean([r["elapsed_seconds"] for r
in runs]))}
        rows.append(row)
    with (output / "summary.csv").open("w", newline="") as stream:
        writer = csv.DictWriter(stream, fieldnames=list(rows[0]))
        writer.writeheader(); writer.writerows(rows)
    (output / "summary.json").write_text(json.dumps(rows, indent=2))
    figures = output / "figures"
    figures.mkdir(exist_ok=True)
    panels = [("Crossover probability", [f"pc_{p:.1f}" for p in (0, .2, .5, .8, 1)]),
              ("Selection operator", ["pc_0.8", "select_roulette", "select_rank"]),
              ("Crossover operator", ["pc_0.8", "cross_one_point", "cross_uniform"]),
              ("Mutation operator", ["pc_0.8", "mut_adaptive"])]
    fig, axes = plt.subplots(2, 2, figsize=(12, 8), constrained_layout=True)
    for ax, (title, keys) in zip(axes.flat, panels):
        for key in keys:
            curves = np.array([[h["best"] for h in r["history"]] for r in groups[key]])
            mean, sd = curves.mean(axis=0), curves.std(axis=0, ddof=1)
            ax.plot(mean, label=LABELS[key])
            ax.fill_between(range(len(mean)), mean - sd, np.minimum(100, mean + sd), alpha=.12)
        ax.set(title=title, xlabel="Generation", ylabel="Best fitness (mean +/- SD)", ylim=(55, 101))
        ax.legend(fontsize=8); ax.grid(alpha=.2)
    fig.savefig(figures / "convergence.png", dpi=180); plt.close(fig)
    fig, axes = plt.subplots(2, 1, figsize=(11, 7), constrained_layout=True)
    names = [LABELS[r["config_id"]] for r in rows]
    colors = ["#3567aa" if r["config_id"] != "pc_0.8" else "#d67526" for r in rows]
    axes[0].bar(names, [r["success_rate"] * 100 for r in rows], color=colors)
    axes[0].set(ylabel="Success rate (%)", ylim=(0, 110), title=f"Fixed budget; {len(baseline)} paired seeds")
    axes[1].bar(names, [r["penalized_hit_mean"] for r in rows],
                yerr=[r["penalized_hit_std"] for r in rows], capsize=3, color=colors)
    axes[1].set(ylabel="Penalized hitting generation", title="Failure assigned G+1; lower is better; error bars
= SD")
    for ax in axes:
        ax.tick_params(axis="x", rotation=30); ax.grid(axis="y", alpha=.2)
```

```
    fig.savefig(figures / "performance.png", dpi=180); plt.close(fig)
    fig, axes = plt.subplots(1, 2, figsize=(10, 3.6), constrained_layout=True)
    for key in ("pc_0.8", "mut_adaptive"):
        for ax, metric in zip(axes, ("diversity", "pm")):
            curves = np.array([[h[metric] for h in r["history"]] for r in groups[key]])
            ax.plot(curves.mean(axis=0), label=LABELS[key])
    axes[0].set(xlabel="Generation", ylabel="Normalized gene diversity")
    axes[1].set(xlabel="Generation", ylabel="Per-bit mutation probability (next step)")
    for ax in axes: ax.legend(); ax.grid(alpha=.2)
    fig.savefig(figures / "mutation_dynamics.png", dpi=180); plt.close(fig)


if __name__ == "__main__":
    import argparse
    parser = argparse.ArgumentParser()
    parser.add_argument("results", type=Path)
    summarize(parser.parse_args().results)
```

验证代码（test_ga.py）

```
import unittest
from dataclasses import replace
import numpy as np
from ga import Config, crossover, diversity, mutation_probability, run, select_indices


class GeneticAlgorithmTests(unittest.TestCase):
    def test_crossover_preserves_alleles_and_parents(self):
        parents = np.array([[0]*20,[1]*20]*10,dtype=np.uint8)
        original = parents.copy()
        for kind in ('one_point','two_point','uniform'):
            off = crossover(parents,kind,1,np.random.default_rng(0))
            np.testing.assert_array_equal(off[::2]+off[1::2],parents[::2]+parents[1::2])
            self.assertTrue(np.isin(off,[0,1]).all())
            np.testing.assert_array_equal(parents,original)
            np.testing.assert_array_equal(crossover(parents,kind,0,np.random.default_rng(0)),parents)

    def test_run_reproducibility_budget_and_elitism(self):
        config=Config(generations=30,population=20,length=20)
        for mutation in ('basic','adaptive'):
            c=replace(config,mutation=mutation)
            a,b=run(c,123),run(c,123)
            a.pop('elapsed_seconds'); b.pop('elapsed_seconds')
            self.assertEqual(a,b)
            self.assertEqual(a['evaluations'],20*31)
            self.assertEqual(a['final_best'],a['best_genome'].count('1'))
            best=[h['best'] for h in a['history']]
            self.assertTrue(all(y>=x for x,y in zip(best,best[1:])))

    def test_selection_edge_cases(self):
        fitness=np.zeros(80)
        for kind in ('roulette','rank','tournament'):
            ids=select_indices(fitness,replace(Config(),selection=kind),np.random.default_rng(5))
            self.assertEqual(ids.shape,(80,))
            self.assertTrue(((ids>=0)&(ids<80)).all())
        f=np.array([0,0,0,10])
        ids=select_indices(f,replace(Config(population=4),selection='roulette'),np.random.default_rng(0))
        self.assertTrue((ids==3).all())

    def test_adaptation_bounds(self):
```

```
        c=replace(Config(),mutation='adaptive')
        self.assertAlmostEqual(diversity(np.array([[0,0],[1,1]])),1)
        self.assertAlmostEqual(diversity(np.ones((4,5))),0)
        self.assertAlmostEqual(mutation_probability(c,1),.01)
        self.assertAlmostEqual(mutation_probability(c,0),.03)

    def test_configs_and_shared_initial_population(self):
        base=run(Config(generations=1),99)
        for c in (replace(Config(),pc=0,generations=1),replace(Config(),selection='rank',generations=1)):
            self.assertEqual(base['initial_population'],run(c,99)['initial_population'])
        for c in (Config(pc=2),Config(population=5),Config(selection='bad')):
            with self.assertRaises(ValueError): run(c)


if __name__=='__main__': unittest.main()
```

运行环境与依赖（pyproject.toml）

```
[project]
name = "ga-group-homework"
version = "0.1.0"
description = "Reproducible OneMax genetic algorithm operator experiments"
requires-python = ">=3.12"
dependencies = ["deap==1.4.3", "numpy==2.5.4", "matplotlib==3.10.8"]

[tool.uv]
package = false
```

实验结果

原始示例完整运行输出

来源：results/original_stdout.txt。

```
gen nevals  avg     std     min max
0   300     50.4933 5.33822 34  64
1   181     54.8333 4.34958 44  66
2   191     58.4567 3.45564 47  68
3   199     60.9533 2.9025  52  68
4   167     62.96   2.90719 47  71
5   175     64.99   2.84896 57  73
6   168     66.9333 2.80515 58  74
7   187     68.9167 2.82661 59  76
8   171     70.8867 2.4455  62  76
9   155     72.69   2.62435 62  80
10  171     74.1233 2.61052 64  82
11  191     75.64   2.70007 65  82
12  171     77.18   2.55752 69  84
13  173     78.7667 2.24475 69  84
14  185     79.9067 2.36459 72  86
15  205     81.4433 2.3805  72  88
16  163     82.6767 2.22534 74  88
17  175     83.6833 2.37411 76  88
18  181     84.8067 2.30274 74  90
```

```
 19  179     85.6233 2.51955 74  91
 20  178     86.58   2.16416 78  91
 21  173     87.2533 2.33148 78  91
 22  155     88.06   2.15787 79  92
 23  187     88.37   2.20146 80  92
 24  184     89.2767 1.97825 82  94
 25  198     89.7767 2.3805  80  95
 26  185     90.6233 2.41553 80  96
 27  160     91.62   2.25291 82  96
 28  182     92.45   2.36379 83  97
 29  171     93.2933 2.46589 84  97
 30  184     94.1433 2.39919 84  97
 31  161     94.91   2.40594 85  98
 32  181     95.4633 2.28954 85  99
 33  177     96.02   2.40962 88  99
 34  182     96.7733 2.09172 88  99
 35  177     97.0433 2.32554 86  100
 36  161     97.3567 2.50122 88  100
 37  178     97.9167 2.34302 90  100
 38  176     98.4    2.11345 87  100
 39  202     98.2467 2.61007 88  100
 40  180     98.8333 2.11003 90  100
```

扩展实验统计数据

来源：results/summary.csv；共 10 种配置，每种配置 30 次运行。

数值保留 4 位小数，耗时保留 6 位小数；完整精度数据见原 CSV。成功代数均值仅统计成功运行，无成功运行时记为“—”。

末代最优适应度

配置	次数	均值	标准差	最小值	最大值
pc_0.0	30	99.6667	0.4795	99	100
pc_0.2	30	100.0000	0.0000	100	100
pc_0.5	30	100.0000	0.0000	100	100
pc_0.8	30	100.0000	0.0000	100	100
pc_1.0	30	100.0000	0.0000	100	100
select_roulette	30	86.8333	2.1023	83	91
select_rank	30	99.7333	0.4498	99	100
cross_one_point	30	100.0000	0.0000	100	100
cross_uniform	30	100.0000	0.0000	100	100
mut_adaptive	30	99.1333	0.6814	98	100

成功率与到达代数

配置	成功次数	成功率	成功代数均值	惩罚代数均值	惩罚代数标准差
pc_0.0	20	66.67%	92.8000	95.5333	5.8412
pc_0.2	30	100.00%	65.9667	65.9667	9.0952
pc_0.5	30	100.00%	51.5667	51.5667	5.1641
pc_0.8	30	100.00%	41.2333	41.2333	5.7816
pc_1.0	30	100.00%	37.4333	37.4333	3.8568
select_roulette	0	0.00%	—	101.0000	0.0000
select_rank	22	73.33%	85.2727	89.4667	10.4312
cross_one_point	30	100.00%	47.3667	47.3667	4.0042
cross_uniform	30	100.00%	27.0333	27.0333	4.1313
mut_adaptive	9	30.00%	78.2222	94.1667	13.9261

配对差异与耗时

配置	配对均值差/代	95%区间下限	95%区间上限	平均耗时/秒
pc_0.0	54.3000	51.7000	56.8333	0.022670
pc_0.2	24.7333	21.2667	28.5333	0.022844
pc_0.5	10.3333	7.9000	13.0675	0.022461
pc_0.8	0.0000	0.0000	0.0000	0.022579
pc_1.0	-3.8000	-6.2000	-1.4000	0.022421
select_roulette	59.7667	57.7000	61.7667	0.025988
select_rank	48.2333	43.9667	52.3333	0.033830
cross_one_point	6.1333	3.4667	8.6667	0.020235
cross_uniform	-14.2000	-16.6000	-11.6667	0.024073
mut_adaptive	52.9333	47.1658	58.0000	0.022885